

Cartesian Genetic Programming with Crossover for Designing Combinational Logic Circuits

José Eduardo H. da Silva

*Universidade Federal de Juiz de Fora
Juiz de Fora, MG, Brazil
jehenriques@ice.ufjf.br*

Heder S. Bernardino

*Universidade Federal de Juiz de Fora
Juiz de Fora, MG, Brazil
heder@ice.ufjf.br*

Abstract—The development of an efficient crossover for Cartesian Genetic Programming (CGP) has been widely investigated, but there is not a large number of approaches using this type of operator when designing combinational logic circuits. In this paper, we introduce a new crossover for CGP when using a single genotype representation and the desired model has multiple outputs. The proposal modifies the standard evolutionary strategy commonly adopted in CGP by combining the subgraphs of the best outputs of the parent and its offspring in order to generate a new fittest individual. The proposed crossover is applied to combinational logic circuits with multiple outputs, a parameter analysis is performed, and the results obtained are compared to those found by a baseline CGP and other techniques from the literature.

1. Introduction

The project and development of electronic circuits is a process with huge complexity that demands time, experience and knowledge of collections of rules, mainly when the circuits are large, with lots of inputs and outputs. These circuits are very important and they are present in most electro-electronic devices nowadays. Among the different kinds of digital circuits one is the combinational logic circuits (CLCs) and its project is based on the information given by a truth table that lists all possible combinations of the inputs with their respective outputs.

In order to assist the designers, several methods and traditional techniques are commonly used, such as Espresso [1] and Quine-McCluskey method [2]. Further on those traditional techniques the evolutionary computation has been shown to be quite efficient in different applications for evolving circuits [3], [4], [5], [6], [7]. Recently, the so called Evolvable Hardware (EH) has been questioned as its techniques are not showing to be competitive when the real needs of practical applications are considered. The main justification is the scalability of these techniques, which makes hard the evolution of more complex circuits [8].

Currently, Cartesian Genetic Programming (CGP) [9] is one of the most efficient method for the evolutionary design and optimization of CLCs [10], despite other approaches still being used [6], [7]. CGP is a genetic programming

technique where programs are modeled as directed acyclic graphs (DAGs) and, thus, a large number of computational structures can be easily represented, such as CLCs [11]. Despite its widespread use for the evolution of digital circuits, CGP has difficulty when working with more complex circuits. The 4-bit multiplier is the most complex circuit already evolved [12], [13]. The evolution of more complex circuits using CGP can be found only minimizing the number of gates of known models [10].

CGP normally uses only mutation to generate new individuals, and the point mutation and the SAM (Single Active Mutation) are the most adopted approaches. Further, Miller [9] highlights that crossover operators have received little attention in the CGP. Originally a one-point crossover operator was used but it was shown to be disruptive for the subgraphs of CGP and had a detrimental effect on the performance. Other approaches showed improvements in CGP when a crossover operator was used [14], [15], [16], [17]. Even so the mutation operator is still the most used variation operator in CGP.

A new crossover operator for the design of CLCs is proposed here, where the solutions created in each generation are combined in order to generate a better model. Thus, the Evolution Strategy (ES) commonly used in CGP is also modified. In this way, the main objectives of this paper are: (i) to propose a crossover operator to CGP when applied to CLCs, (ii) to compare the results of a baseline CGP and from other methods from the literature with those obtained by using the proposed crossover in terms of reducing the number of evaluations needed to find a feasible solution, and (iii) to analyze the performance of CGP when varying the number of individuals (λ) and the number of active nodes.

2. Related Work

2.1. Cartesian Genetic Programming (CGP)

Miller [9] proposed a new form of genetic programming, called Cartesian Genetic Programming. Despite his initial intention of evolving digital circuits, CGP is nowadays applied in several areas, such as robot controllers [5], neural networks [18], and image classifiers [19].

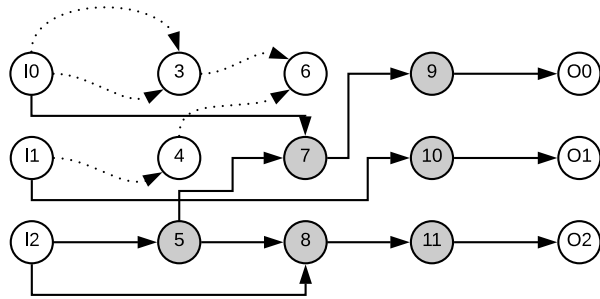


Figure 1: Example of a CGP individual with *levels-back=number of column*. Inactive nodes are in white and the continuous lines define the phenotype.

In CGP, the programs are DAGs represented by a matrix of processing nodes and each node contains genes that describe its function and connections to the other nodes. Each node connects to other ones on its left columns in the matrix, as shown in Figure 1. This figure illustrates a CGP individual with a 2×4 matrix when solving a problem with 3 inputs and 2 outputs. CGP has three user-defined parameters: number of columns (n_c) and rows (n_r) of the matrix that encodes the graph, and levels-back (lb). Levels-back is the number of columns (on the left size) in which each node can be connected.

The most common search technique used in CGP is an Evolution Strategy (ES) $(1 + \lambda)$ [9], where λ is the number of new solutions generated at each iteration of the algorithm. In this case, the best individual between the parent and the λ new generated solutions is selected to the next generation.

In CGP, the most used mutation operator is the point mutation [9]. However, when using the point mutation, the changes may occur in inactive nodes leading to a lack of modifications in the phenotype.

These nodes which do not interfere in the output of the solutions are called *inactive nodes* (white nodes in Figure 1). The single active mutation (SAM) [18], [19] is another common mutation operator, where (i) one node and one of its elements (inputs or gate) are selected at random, and (ii) the value is changed to another valid value. The steps (i) and (ii) are repeated until an active node is modified. So SAM ensures that one active node is changed. Although SAM may modify more than one active node, there is no study in the literature investigating this parameter. In fact, it is suggested as a future work in [18]. Here we analyze SAM when multiple active genes are changed; this proposal is labeled here as Multiple Active Mutation (MAM).

2.2. Embedded Cartesian Genetic Programming

Embedded Cartesian Genetic Programming (ECGP) was developed by Walker and Miller [20] and incorporates ideas from a technique known as Module Acquisition [21] to CGP. This allows for the automatic re-use of partial solutions [16].

ECGP's genotype is a list of integers that encodes the connections and functions of each node of the DAG. However, each module in ECGP can change its length: operations

of compression and expansion. ECGP encodes each gene with a pair of integers, where the first integer pair represents (i) the primitive function or module that the node represent, and (ii) its type [16]. More details regarding ECGP can be found in [20], [22], [23].

2.3. Multi-Chromosome Approaches

When a multi-chromosome approach is used, the genotype is divided into two or more blocks of equal length. The number of chromosomes present in the genotype of an individual is dictated by the number of program outputs required by the problem, as each chromosome is connected to a single program output. Each chromosome contains an equal number of nodes, and is treated as a genotype of an individual with single program output. The inputs of each node encoded in a chromosome are only allowed to connect to the output of earlier nodes encoded in the same chromosome or any program input [16].

2.4. Crossover in CGP

Clegg et al. [15] introduced four variants of a crossover operator for regression problems using an integer representation. Those four variants failed to improve the convergence of CGP. After that, they presented another crossover where CGP represents the DAG as a fixed length list of real-valued numbers in the interval $[0, 1]$. Recombination is performed using an arithmetic crossover with a random weighting factor, as used within real-valued genetic algorithms. This last crossover operator performed better than the 4 previously ones but the comparative analysis is inconclusive when it is compared to the traditional CGP.

Kalkreuth [17] developed a crossover inspired by that in the tree-based Genetic Programming [24]. The suitable crossover point is defined by selecting two crossover points (one in each parent) considering the active nodes and the length of the paths. Only the genetic material of the active paths are recombined and one offspring is generated. The computational experiments were composed by symbolic regression, image operator design, and Boolean functions. That method obtained better results than those of the arithmetic crossover, and presented a smaller number of objective function evaluations to finish.

CGP is positional dependent, as the effect or meaning of a component in the program is determined by its absolute position in the genotype. The manner in which components are referenced in CGP is considered arbitrary as there is no correlation between a component's absolute coordinates and its behaviour. Cai [25] achieved a positive effect of crossover in CGP using an implicit context representation in which recombination is useful for the Even Parity-3 problem.

Walker et al. [16] investigated a crossover for multi-chromosome representations. In that case, multiple output circuits were represented using one graph for each output, and a new individual is generated combining the graphs with the best outputs. The results obtained by that technique

were compared to those found by the traditional single-chromosome CGP and ECGP, and performed better than the other ones, specially when solving problems with several outputs and an independent fitness assignment.

Recently, Husa [26] made a comparative study of crossover in CGP and proposed a new operator, called Block Crossover. Inspired by the cone-based crossover for ECGP of [14], that was applied in Boolean functions, the idea is to extend the operator to standard CGP. The block contains a desired number of nodes and all nodes in the block are directly linked through their inputs or outputs. Also, all nodes in the block are part of the genotype's active path. However, the block crossover did not present better results than other approaches to the subgraph and arithmetic crossover.

3. The Proposed Crossover

The proposed crossover operator consists of recombining the individuals with the best output values, considering the parent and its offspring. The main advantage of the proposed method is the use of parts of the candidate solutions which produces the best results considering the parent and its offspring to improve the best individual of the current population. This approach is similar to that proposed by Walker et al. [16], but here a single chromosome is evolved. Thus, this single chromosome represents the entire circuit and the active genes may be shared to compute the outputs, resulting in a more compact circuit. As a result, the number of gates is smaller than that in a solution found when using multiple chromosomes, improving the quality of the solutions with respect to a common objective in the design of circuits. Despite this advantage, here we are not optimizing the number of gates and, thus, it is not expected a minimum number of gates in the solution.

The proposed crossover creates an individual that is composed by the subgraphs that encode the best outputs present in the individuals of the current population. For the proposed crossover, the individuals with the greatest number of correct values for each output, which are those ones with the largest number of correct hits with respect to the truth table, are selected (*best_inds*). The base individual (*best_ind*) is that, from these selected ones, with the maximum number of best outputs. With this candidate solution, it is possible to determine which outputs will be recombined. The subgraphs of the base individual which are not in the best-selected individuals are then replaced by those subgraphs which generate the best outputs. One can notice that the number of matches of the truth table and the outputs of the individual created by the proposed crossover is larger than or equals to that observed by the parent and its offspring. Consequently, this newly created individual is better than or equals to any other candidate solution found during the search.

The crossover operator takes into account the position of the nodes that should be incorporated into the base individual. Nothing is done when the node to be incorporated is equal to that in the base individual. Otherwise, this node

TABLE 1: Matches with respect to the truth table

Individual	O0	O1	O2	Fitness
1	7	3	6	16
2	5	8	4	17

replaces the first available inactive node. The entries of the remaining nodes to be incorporated are modified according to the new position of that node. Algorithm 1 shows a pseudo-code of the proposed crossover.

Algorithm 1 The proposed crossover.

```

1: best_inds ← individuals with the maximum number of
   correct values for each output i
2: best_ind ← most frequent individual in best_inds
3: for each output i do
4:   if best_ind is not best_indsi then
5:     active_path ← all nodes of the active path of
       the i-th output of best_indsi
6:     for each node index k ∈ active_path do
7:       if node k ≠ k-th node of best_ind then
8:         available_node ← index of first inactive
           node of best_ind after index k
9:         best_indavailable_node ← node k
10:      end if
11:    end for
12:  end if
13: end for

```

Two individuals with three inputs (I0, I1, and I2) and three outputs (O0, O1, and O2) are used here to illustrate the proposed crossover, as shown in Figure 2 (a) and (b). The numbered circles represent the nodes of the individual. The nodes in gray are active, and the white ones are inactive. The arrows represent the directed connections between the nodes. In this way, the path to the output O2 of the first individual is formed by I2, and the nodes 8 and 11. In this example, each output matches the truth table according to Table 1.

The first individual has the best O0 and O2 outputs. Output O1 is best in the second individual. Thus, the base individual of the crossover is the first one, and the path to the output O1 of the second individual will be incorporated into this base individual. The individual created by crossover is shown in Figure 2 (c), where the gray nodes are the original nodes of the base individual (first individual) and the blue node is the node in the path of the output O1 of the second individual. The dashed lines form the new path in order to calculate the output O1 of the new individual.

One can observe in Table 1 that the standard CGP chooses the second individual as the best individual of the population, as it has 17 matches with respect to the truth table. In CGP, this individual survives to the next generation. On the other hand, the proposed crossover approach generates a new individual which hits 21 of the 24 values.

The proposed crossover does not require any additional parameter or crossover rate and it is applied during the selection step of the evolution strategy. Once it is guaranteed

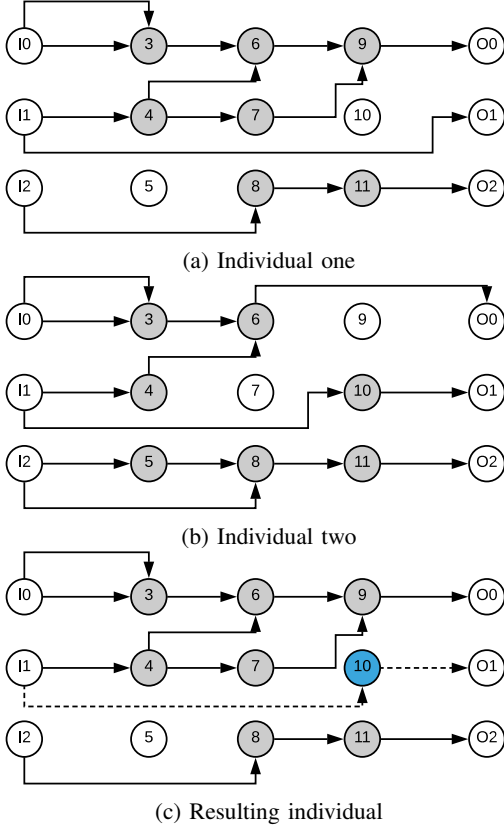


Figure 2: (a) and (b) are two individuals from the population. (c) is the new individual generated by the proposed crossover; the output O1 of the second individual (b) is incorporated to the genotype of the first one (a).

that the new individual will be the best or equivalent to the best individual in the population, no tournament selection is needed. When the population contains two or more individuals which are considered the best one and there are no outputs to be incorporated in the best individual, the algorithm chooses one of them randomly. When the problem contains only a single output, the proposed approach will select the individual with the best fitness, such as the standard CGP. Thus, this approach is mainly suitable for problems with multiple outputs. In addition, inactive nodes are present throughout the genotype, so that the neutral genetic drift can occur.

4. Computational Experiments

Computational experiments were performed in order to comparatively analyze the performance of the proposed crossover technique (labeled here as X-CGP). First, we compare the results obtained by X-CGP with those from the literature. In the second experiment, the performance of X-CGP is analyzed with different values for the number of offspring λ and active nodes.

TABLE 2: Problems used in Experiment 1.

Problem	Inputs	Outputs	nc	Max. Eval.
2-bit Adder	5	3	100	100,000
3-bit Adder	7	4	500	1,000,000
2-bit Multiplier	4	4	100	100,000
3-bit Multiplier	6	6	500	1,000,000
3:8-bit De-multiplexer	3	8	100	100,000
4x1-bit Comparator	4	18	1500	1,000,000

4.1. Experiment 1

Here, X-CGP is compared to a baseline CGP and to the techniques presented in [16], where a crossover is also used in CGP. Computational effort [9], median (ME), median absolute deviation (MAD) and interquartile range (IQR) of the number of objective function evaluations to find a feasible solution are used to compare the results obtained. The 7 problems used in [9], [16] are considered here and these circuits are described in Table 2. The exception is the Arithmetic Logic Unit (ALU), as the proposed approach did not obtained a feasible solution for this problem. The 3:8-bit De-multiplexer uncompress a compressed 3-bit signal into its eight components and the 4x1-bit comparator compares all combinations of inputs for $<$, $=$ and $>$. The remaining circuits are arithmetic ones, designed to add or multiply binary encoded values. We adopted $\lambda = 4$, and $n_r = 1$ and $lb = n_c$. Thus, the genotype can represent any DAG with the number of nodes equals to the number of columns [9]. The number of columns and the maximum number of evaluations are also presented in Table 2. The number of independent runs is 30 for all the tested problems, except to the 3-bit Multiplier where 10 independent runs were performed due to its complexity. For all problems we used the function set $\Gamma = \{\text{AND}, \text{OR}, \text{NOT}, \text{XOR}, \text{WIRE}\}$. We also analyzed the performance of the proposed crossover when using point mutation and SAM. As SAM found the best circuits in the preliminary experiments, the results obtained when using point mutation are not presented here. The source-code of the proposal is available¹.

Feasible solutions were obtained by X-CGP in all the independent runs, except for ALU. It is important to notice that the methods using a single chromosome in [16] were also unable to find feasible solutions for this problem.

The computational efforts observed for all the tested techniques are presented in Table 3, as well as the speedups (the computational effort of a given technique divided by that obtained by X-CGP). The best computational efforts are highlighted in boldface. The experiments showed that X-CGP outperforms the other single-chromosome techniques. Considering these cases, the proposed approach obtained a speedup from 1.27 to 22.70. Also, the proposed approach performed better than the other techniques considered here in the adder problems. On the other hand, the multi-chromosome approaches obtained the lowest computational effort in 4 of the 6 problems, and the speedups are from 0.05 to 4.43 considering MC-CGP and MC-ECGP.

1. <https://github.com/ciml/ciml-lib>

ME, MAD and IQR values of the single and multi-chromosome approaches in [16] are presented in Table 4 and those obtained by the proposed approach are presented in Table 5. X-CGP performed better than the approaches with single chromosomes in terms of median and median absolute deviation. Also, X-CGP is better than the methods with multi-chromosomes on both adder problems.

4.2. Experiment 2

A parameter analysis is performed in this second experiment, where we analyze the performance of X-CGP when the following values are modified: (i) the number of offspring λ , and (ii) the number of active genes modified by MAM. Values between 1 and 10 are tested.

Five of the most commonly found circuits in the evolutionary computation literature are used here [3], [27]. Table 6 presents the problems and the number of columns, the number of inputs, the number of outputs and the maximum number of objective function evaluations allowed. The parameters n_τ and lb, and the function set are those used in Section 4.1. We performed 30 independent runs for each configuration for each problem, with $n_c = 100$, except for the problem CM85A where 3 independent runs were performed for each configuration and $n_c = 300$ due to the larger number of inputs when compared to the other circuits.

Performance Profiles (PPs) is used to analyze the results and, as suggested in [28], the normalized area under the PP curves (AUC) is adopted as an overall performance measure (larger AUCs are preferable). PPs are an analytical tool for the visualization and interpretation of experimental results. According to the AUCs, the worst results were obtained when using smaller lambda values, such as 1 or 2. On the other hand, the three best parameter settings are: (i) MAM(1); $\lambda = 9$, (ii) MAM(1); $\lambda = 3$, and (iii) MAM(3); $\lambda = 5$. The PPs of these three variants are shown in Figure 3, where one can observe that (i) MAM(1)-L3 and MAM(3)-L5 are the variants with the best performance in most of the problems (largest $\rho(1)$); (ii) MAM(1)-L9 is the most reliable variant (smallest τ such that $\rho(\tau) = 1$); and (iii) MAM(1)-L9 presents the best overall performance (largest AUC). Also, it is important to highlight that among these three variants, MAM(3)-L5 is the only one with success rate (percentage of the independent runs in which a feasible solution is found) equals to 100% for all problems tested.

5. Concluding Remarks and Future Work

A new crossover for Cartesian Genetic Programming (CGP) was introduced here in order to design combinational logic circuits. The proposed crossover was compared to a baseline CGP and other techniques from the literature. We can conclude that the proposed X-CGP (i) obtained the best results when compared to the methods with a single chromosome, and (ii) performed better than the multi-chromosome techniques in some of the problems.

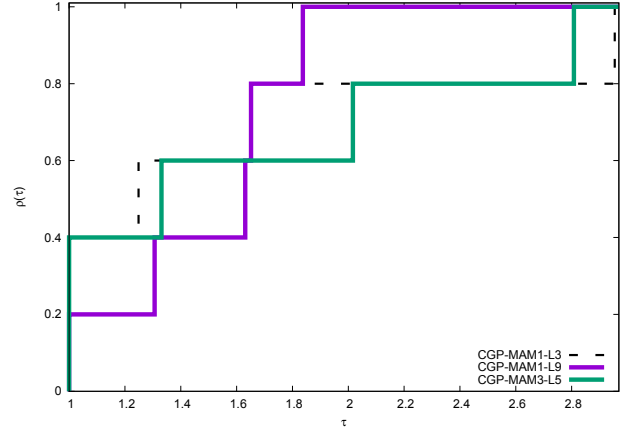


Figure 3: Performance Profiles of the three best X-CGP variants. The normalized AUC are 1.0, 0.9977 and 0.9952 for MAM(1)-L9, MAM(1)-L3 and MAM(3)-L5, respectively.

These results indicate that the proposed method is promising. However, X-CGP performed worse than the multi-chromosome techniques considered here in most of the test-problems, suggesting that a deeper analysis is necessary, especially when additional objectives are considered (for instance, the number of gates).

The parameter setting of metaheuristics is a hard task and an analysis was performed, where we concluded that: (i) 100% success rate and the best performance in most of the problems are obtained when modifying 3 active nodes and generating 5 offspring; and (ii) the variant where 1 active node is modified and 9 offspring are generated is more reliable and found the best overall results.

Finally, we also intend to apply the proposed crossover to even more complex problems.

Acknowledgments

The authors thank the financial support provided by FAPEMIG, CNPq, CAPES, and PPGCC.

References

- [1] R. K. Brayton, G. D. Hachtel, C. McMullen, and A. Sangiovanni-Vincentelli, *Logic minimization algorithms for VLSI synthesis*. Springer Science & Business Media, 1984, vol. 2.
- [2] E. J. McCluskey, "Minimization of boolean functions," *Bell Labs Technical Journal*, vol. 35, no. 6, pp. 1417–1444, 1956.
- [3] C. Coello, A. Aguirre, and B. Buckles, "Evolutionary multiobjective design of combinational logic circuits," in *Proc. of the 2nd NASA/DoD Workshop on Evolvable Hardware*. IEEE, 2000, pp. 161–170.
- [4] C. A. C. Coello and A. H. Aguirre, "Design of combinational logic circuits through an evolutionary multiobjective optimization approach," *AI EDAM*, vol. 16, no. 1, pp. 39–53, 2002.
- [5] B. M. García and C. A. C. Coello, "An approach based on the use of the ant system to design combinational logic circuits," *Mathware and Soft Computing*, vol. 9, no. 2-3, pp. 235–250, 2002.

TABLE 3: Computational efforts and speedups (the ratios between the values of X-CGP and those ones of the other methods).

Problem	CGP (1)	ECGP (2)	MC-CGP (3)	MC-ECGP (4)	X-CGP (5)	Speedup (1) Vs. (5)	Speedup (2) Vs. (5)	Speedup (3) Vs. (5)	Speedup (4) Vs. (5)
2-bit Adder	469,200	311,200	140,800	242,000	54,661	8.58	5.69	2.58	4.43
3-bit Adder	8,190,400	2,166,000	1,286,000	1,230,400	360,746	22.70	6.00	3.56	3.41
2-bit Multiplier	52,000	42,000	11,200	22,400	32,962	1.58	1.27	0.34	0.68
3-bit Multiplier	18,509,600	7,103,600	873,600	867,600	950,374	19.48	7.48	0.92	0.91
3:8-bit De-multiplexer	75,200	48,400	4,400	6,400	24,326	3.09	1.99	0.18	0.26
4x1-bit Comparator	3,922,000	1,548,600	10,000	19,200	199,086	19.7	7.78	0.05	0.10

TABLE 4: Results obtained by the baseline CGP, ECGP and the multi-chromosome approaches.

Problem	CGP			ECGP			MC-CGP			MC-ECGP		
	ME	MAD	IQR	ME	MAD	IQR	ME	MAD	IQR	ME	MAD	IQR
2-bit Adder	132,565	76,228	178,335	124,251	49,926	99,291	29,239	13,350	25,663	43,117	21,330	50,734
3-bit Adder	1,943,585	996,482	2,174,500	733,909	307,694	599,132	174,625	82,050	222,178	222,889	90,642	199,633
2-bit Multiplier	6,197	4,130	8,489	7,485	3,304	9,875	1,721	662	1,872	2,809	1,410	3,856
3-bit Multiplier	4,030,201	2,181,656	6,110,863	854,579	257,354	773,789	140,643	52,722	102,960	129,767	54,178	97,426
3:8-bit De-mux	13,797	4,842	9,635	7,809	3,210	7,618	845	274	524	985	402	1,103
4x1-bit Comp.	770,475	335,916	672,160	271,553	93,906	178,257	1,693	704	1,442	3,097	1,620	3,156

TABLE 5: Results of X-CGP.

Problem	ME	MAD	IQR
2-bit Adder	9,012	5,797.5	17,913
3-bit Adder	87,476.5	54,985.5	122,575.75
2-bit Multiplier	6,489.5	3,098	8,487
3-bit Multiplier	288,182	129,432.5	426,209.75
3:8-bit De-multiplexer	6,887	2,410	5,412
4x1-bit Comparator	31,308.5	22,117.5	76,231

TABLE 6: Problems used for the parameters adjustment

Problems	Inputs	Outputs	nc	Max. Eval.
1 - Example 4 [3]	4	3	100	100,000
2 - 2x2-bit Multiplier	4	4	100	100,000
3 - Example 4 [27]	4	2	100	100,000
4 - 2x2-bit Adder	4	4	100	100,000
5 - CM85A	8	3	300	1,000,000

- [6] E. Stomeo, T. Kalganova, C. Lambert, N. Lipnitsakya, and Y. Yatskevich, "On evolution of relatively large combinational logic circuits," in *NASA/DoD Conf. on Evolvable Hardware*, 2005, pp. 59–66.
- [7] S. Karakatic, V. Podgorelec, and M. Hericko, "Optimization of combinational logic circuits with genetic programming," *Elektronika ir Elektrotehnika*, vol. 19, no. 7, pp. 86–89, 2013.
- [8] S. Stepney and A. Adamatzky, *Inspired by Nature: Essays Presented to Julian F. Miller on the Occasion of His 60th Birthday*. Springer, 2017, vol. 28.
- [9] J. F. Miller, "Cartesian genetic programming," *Cartesian Genetic Programming*, pp. 17–34, 2011.
- [10] Z. Vasicek, "Cartesian gp in optimization of combinational circuits with hundreds of inputs and thousands of gates," in *European Conference on Genetic Programming*. Springer, 2015, pp. 139–150.
- [11] J. F. Miller and S. L. Smith, "Redundancy and computational efficiency in cartesian genetic programming," *IEEE Trans. on Evolutionary Computation*, vol. 10, no. 2, pp. 167–174, 2006.
- [12] V. K. Vassilev, D. Job, and J. F. Miller, "Towards the automatic design of more efficient digital circuits," in *Proc. of the NASA/DoD Workshop on Evolvable Hardware*. IEEE, 2000, pp. 151–160.
- [13] Z. Vasicek, "Bridging the gap between evolvable hardware and industry using cartesian genetic programming," in *Inspired by Nature*. Springer, 2018, pp. 39–55.
- [14] P. Kaufmann and M. Platzner, "Advanced techniques for the creation and propagation of modules in cartesian genetic programming," in *Proc. of the Conference on Genetic and Evolutionary Computation*. ACM, 2008, pp. 1219–1226.
- [15] J. Clegg, J. A. Walker, and J. F. Miller, "A new crossover technique for cartesian genetic programming," in *Proc of the Conf. on Genetic and Evolutionary Computation*. ACM, 2007, pp. 1580–1587.
- [16] J. A. Walker, J. F. Miller, and R. Cavill, "A multi-chromosome approach to standard and embedded cartesian genetic programming," in *Proceedings of the 8th annual conference on Genetic and evolutionary computation*. ACM, 2006, pp. 903–910.
- [17] R. Kalkreuth, G. Rudolph, and A. Droschinsky, "A new subgraph crossover for cartesian genetic programming," in *European Conference on Genetic Programming*. Springer, 2017, pp. 294–310.
- [18] B. W. Goldman and W. F. Punch, "Reducing wasted evaluations in cartesian genetic programming," in *European Conference on Genetic Programming*. Springer, 2013, pp. 61–72.
- [19] —, "Analysis of cartesian genetic programming's evolutionary mechanisms," *IEEE Trans. on Evolutionary Computation*, vol. 19, no. 3, pp. 359–373, 2015.
- [20] J. A. Walker and J. F. Miller, "Evolution and acquisition of modules in cartesian genetic programming," in *European Conference on Genetic Programming*. Springer, 2004, pp. 187–197.
- [21] P. J. Angeline and J. Pollack, "Evolutionary module acquisition," in *Proceedings of the second annual conference on evolutionary programming*, 1993, pp. 154–163.
- [22] J. A. Walker and J. F. Miller, "Improving the evolvability of digital multipliers using embedded cartesian genetic programming and product reduction," in *International Conference on Evolvable Systems*. Springer, 2005, pp. 131–142.
- [23] —, "Investigating the performance of module acquisition in cartesian genetic programming," in *Proc. of the Conference on Genetic and Evolutionary Computation*. ACM, 2005, pp. 1649–1656.
- [24] J. R. Koza, *Genetic programming: A paradigm for genetically breeding populations of computer programs to solve problems*. Stanford University, Dept. of Computer Science, Stanford, CA, 1990, vol. 34.
- [25] X. Cai, S. L. Smith, and A. M. Tyrrell, "Positional independence and recombination in cartesian genetic programming," in *European Conference on Genetic Programming*. Springer, 2006, pp. 351–360.
- [26] J. Husa and R. Kalkreuth, "A comparative study on crossover in cartesian genetic programming," in *European Conference on Genetic Programming*. Springer, 2018, pp. 203–219.
- [27] C. C. Coello, E. H. Luna, and A. H. Aguirre, "A comparative study of encodings to design combinational logic circuits using particle swarm optimization," in *Proc. of NASA/DoD Conference on Evolvable Hardware*. IEEE, 2004, pp. 71–78.
- [28] H. J. Barbosa, H. S. Bernardino, and A. M. Barreto, "Using performance profiles to analyze the results of the 2006 cec constrained optimization competition," in *Evolutionary Computation (CEC), 2010 IEEE Congress on*. IEEE, 2010, pp. 1–8.